Họ và tên: Nguyễn Hoàng Phúc

MSSV: 24IT206

CHƯƠNG 2

Bài 1:

Câu 3:

1. Thực hiện 3 bước để tìm max, min:
2. So sánh từng cặp phần tử, phần tử nhỏ hơn đặt ở vị trí chỉ số lẻ, phần tử lớn hơn đặt ở vị trí chỉ số chẵn.
3. Tìm phần tử nhỏ nhất trong số các phần tử có chỉ số lẻ (lưu ý, trường hợp n lẻ, phần tử thứ n sẽ không được so sánh ở bước 1).
4. Tìm phần tử lớn nhất trong số các phần tử có chỉ số chẵn. (Viết mã giả thuật toán maxmin(A, n)).
5. Đánh giá số phép so sánh C(n) của thuật toán ở Câu 3.

Câu 4:

Đánh giá số phép so sánh C(n) của thuật toán ở Câu 3.

Bài 2:

Đánh giá độ phức tạp

Gọi C(n) là số phép so sánh trong trường hợp xấu nhất của thuật toán tìm kiếm trên mảng có n phần tử.

Giả sử n = 2^k.

Từ thuật toán, có hệ thức truy hồi:

$$C(n) = \begin{cases} 1 & if \ \ n = 1 \\ C(n/2) + 1 & else \end{cases}$$

Lời giải:

Bài 1:

Câu 3:

```
maxmin (A, n)
begin
  i = 1
  while (i <= n - 1) do
```

```
        if (A[i] > A[i+1]) then hoandoi(A[i], A[i+1]) endif
        i = i + 2
    endwhile
    max = A[1]; i = 3
    while (i <= n) do
        if (max < A[i]) then max = A[i] endif
        i = i + 2
    endwhile
    min = A[2]; i = 4
    while (i <= n) do
        if (min > A[i]) then min = A[i] endif
        i = i + 2
    endwhile
    if (n lẻ) then
        if (max < A[n]) then max = A[n] endif
    endif
    return (max, min)
end
```

Bài 2:

Áp dụng phương pháp thế để giải hệ thức truy hồi khi n = 2^k

$$C(n) = C\left(\frac{n}{2}\right) + 1$$

Vì n = 2^k , thay điều kiện cơ sở C(1) = 1 vào ta có:

$$C(n) = C(1) + k = 1 + k$$

Thay k = log_2 (n) vào biểu thức thu được:

$$C(n) = \log_2(n) + 1$$

CHƯƠNG 3

Bài 2:

- Câu 1: Thuật toán đệ quy tính ước số chung lớn nhất hai số nguyên dương
- Câu 2: Khử đệ quy usc(x,y)

Bài 3:

- Câu 1: Thuật toán đệ quy in dãy nhị phân tương ứng của một số nguyên
- Câu 2: Khử dệ quy

Lời giải :

Bài 2:

Câu 1:

Thuật toán đệ quy in dãy nhị phân tương ứng của một số nguyên n>=0:

```
usc (x, y)
begin
  if (x = y) then return (x)
  else if (x > y) then
    return (usc(x-y, y))
      else (x < y) then
          return (usc(x, y-x))
      endif
  endif
end
```

Câu 2:

Khử đệ quy:

```
usc_khudequy (x, y)
begin
  while (x != y) do
    if (x > y) then
      x = x - y
    else
      y = y - x
    endif
```

```
    endwhile
    return x
  end
```

Bài 3:

Câu 1:

Thuật toán đệ quy in dãy nhị phân tương ứng của một số nguyên n>=0:

```
daynhiphan (n) // n >= 0
begin
  if (n = 0) then
    do nothing
  else
    daynhiphan (n div 2)
    print (n mod 2)
  endif
end
```

Câu 2:

Khử đệ quy:

```
daynhiphan_khudequy (n)
begin
  if (n = 0) then
    print(0)
    return
  endif
  S = Stack_Rong
  while (n > 0) do
    push(S, n mod 2)
    n = n div 2
  endwhile
  while (S không rỗng) do
    print(pop(S))
```

endwhile

end

CHƯƠNG 4

Bài 1:

- Câu 1: Chứng minh rằng có thể nhân hai đa thức ax+b và cx+d chỉ với 3 phép nhân (gợi ý: một trong những phép nhân (a+b)(c+d))
- Câu 2: Xây dựng hai thuật toán chia để trị nhân hai đa thức với độ phức tạp

$$O(n^{\log_2^3})$$

Lời giải :

Bài 1:

Câu 1:

$$P(x) = (ax + b)(cx + d) = ac \cdot x^2 + (ad + bc)x + bd$$

Nếu thực hiện khai triển thông thường, ta cần 4 phép nhân: ac, ad, bc, và bd.

Để giảm xuống còn 3 phép nhân, ta thực hiện các phép nhân sau:

1. $M_1 = a \cdot c$
2. $M_2 = b \cdot d$
3. $M_3 = (a + b)(c + d)$

Khai triển M_3:

$$M_3 = ac + ad + bc + bd = M_1 + (ad + bc) + M_2$$

Từ đó, suy ra biểu thức hệ số của x là (ad + bc):

$$ad + bc = M_3 - M_1 - M_2$$

Thay các giá trị M_1, M_2, M_3 vào tích P(x):

$$P(x) = M_1 \cdot x^2 + (M_3 - M_1 - M_2)x + M_2$$

Kết luận: Ta chỉ cần thực hiện đúng 3 phép nhân M_1, M_2, M_3 cùng với các phép cộng và trừ để tìm ra đa thức tích. => ĐPCM

Câu 2: Thuật toán chia để trị nhân hai đa thức:

```
Algorithm NhanDaThuc_ChiaDeTri(A, B, n):
    // A, B là mảng hệ số của 2 đa thức kích thước n
    // Output: Mảng hệ số đa thức tích C kích thước (2n - 1)
    if n == 1 then
        return [A[0] * B[0]]
    endif
```

m = n / 2

// 1. Tách đa thức thành 2 phần

A0 = A[0 ... m-1]

A1 = A[m ... n-1]

B0 = B[0 ... m-1]

B1 = B[m ... n-1]

// 2. Tính đa thức tổng

A_sum = A0 + A1 // Cộng từng hệ số tương ứng

B_sum = B0 + B1 // Cộng từng hệ số tương ứng

// 3. Gọi đệ quy 3 bài toán con kích thước n/2

Z = NhanDaThuc_ChiaDeTri(A0, B0, m)

Y = NhanDaThuc_ChiaDeTri(A1, B1, m)

X = NhanDaThuc_ChiaDeTri(A_sum, B_sum, m)

// 4. Tính phần hệ số giữa W = X - Y - Z

W = X - Y - Z

*// 5. Tổng hợp kết quả: C = Y * x^(2m) + W * x^m + Z*

C = Mảng khởi tạo kích thước (2n - 1) với các giá trị 0

Cộng các hệ số của Z vào C từ vị trí chỉ số 0

Cộng các hệ số của W vào C từ vị trí chỉ số m

Cộng các hệ số của Y vào C từ vị trí chỉ số 2m

return C

CHƯƠNG 5

Bài 2:

1. Số Catalan được định nghĩa

$$T(n) = \begin{cases} 1 & n = 1 \\ \sum_{i=1}^{n-1} T(i).T(n-i) & n > 1 \end{cases}$$

Xây dựng thuật toán quy hoạch động tính T_n

2. Ý tưởng

- Lưu trữ các giá trị T(i) vào một mảng, i ∈ [1..n]
- Tính các giá trị T(1), T(2), … theo chiều tăng dần
- Tái sử dụng các giá trị từ T(1) đến T(i-1) khi tính T(i)

$$T(i) = \begin{cases} 1 & i = 1 \\ \sum_{j=1}^{i-1} T(j).T(i-j) & i > 1 \end{cases}$$

Bài 3:

Một băng cát-xét gồm 2 mặt, mỗi mặt có thể ghi được d phút (chẳng hạn d = 30). Một đĩa CD chứa n bài hát có tổng thời lượng m phút (m > d, chẳng hạn m = 78). Bài hát i có thời lượng d_i phút, d_i > 0. Cần chọn các bài hát từ đĩa CD lên các mặt băng cát-xét sao cho tổng thời lượng là lớn nhất. Một bài hát được ghi lên một mặt đĩa hoặc không được ghi.

Gọi time(i, t_1, t_2) là thời lượng lớn nhất có thể ghi lên băng cát-xét, trong đó t_1 (t_2) là thời lượng lớn nhất có thể ghi lên mặt thứ nhất (mặt thứ hai), và chỉ sử dụng i bài hát đầu tiên. Khi đó time(n, d, d) là giải pháp của bài toán.

Lời giải:

Bài 2:

function Catalan_DP(n)

begin

T = mảng kích thước (n + 1)

T[1] = 1

for i từ 2 đến n do

T[i] = 0

for j từ 1 đến i - 1 do

*T[i] = T[i] + T[j] * T[i - j]*

endfor

endfor

return T[n]

end

Bài 3:

a) Công thức truy hồi Quy hoạch động:

Đối với bài hát thứ i (thời lượng d_i), ta có 3 lựa chọn:

Không chọn bài hát i: Thời lượng nhận được là time(i-1, t_1, t_2).

Ghi bài hát i vào mặt 1 (với điều kiện thời lượng còn lại $t_1 >= d_i$): Thời lượng nhận được là time(i-1, t_1 - d_i, t_2) + d_i

Ghi bài hát i vào mặt 2 (với điều kiện thời lượng còn lại $t_2 >= d_i$): Thời lượng nhận được là time(i-1, t_1, t_2 - d_i) + d_i

Công thức tổng quát:

$$time(i, t_1, t_2) = \max \begin{cases} time(i-1, t_1, t_2) \\ time(i-1, t_1 - d_i, t_2) + d_i & \text{nếu } t_1 \geq d_i \\ time(i-1, t_1, t_2 - d_i) + d_i & \text{nếu } t_2 \geq d_i \end{cases}$$

Điều kiện cơ sở:

time(0, t_1, t_2) = 0 với mọi 0 <= t_1 <= d, 0 <= t_2 <= d.

b) Thuật toán:

Algorithm CassetteTape_DP(n, d, D):

// n: số bài hát

// d: dung lượng tối đa mỗi mặt

// D: mảng thời lượng các bài hát [d_1, d_2, ..., d_n]

Khởi tạo bảng 3 chiều time[0..n][0..d][0..d] tất cả các phần tử = 0

for i từ 1 đến n do

for t1 từ 0 đến d do

for t2 từ 0 đến d do

```
            // Lựa chọn 1: Không ghi bài i
            val1 = time[i - 1][t1][t2]

            // Lựa chọn 2: Ghi bài i vào mặt 1
            val2 = 0
            if t1 >= D[i] then
               val2 = time[i - 1][t1 - D[i]][t2] + D[i]
            endif

            // Lựa chọn 3: Ghi bài i vào mặt 2
            val3 = 0
            if t2 >= D[i] then
               val3 = time[i - 1][t1][t2 - D[i]] + D[i]
            endif

            time[i][t1][t2] = max(val1, val2, val3)
         endfor
      endfor
   endfor
return time[n][d][d]
```